Zazr and VAVR Tour

Zazr & VAVR · Replace null-config lookup with Option · Build with Waqar

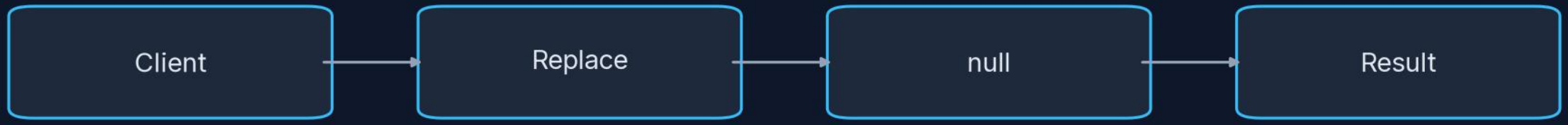
Today's Hook

- Problem: Replace null-config lookup with Option
- Pattern: Zazr & VAVR — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Replace null-config lookup with Option
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Zazr & VAVR: Replace null-config lookup with Option

Real-World Scenario

- Running example: Replace null-config lookup with Option
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```
package com.buildwithwaqar.patterns.functional.vavrcompare;
public final class Demo {
  public static void main(String[] a) {
    System.out.println("=== VAVR vs Zazr walkthrough ===");
    System.out.println("vavrSome=" + VavrVsZazr.vavrDescribe(io.vavr.control.Option.some(1)));
    System.out.println("vavrNone=" + VavrVsZazr.vavrDescribe(io.vavr.control.Option.none()));
    System.out.println("zazrSome=" + VavrVsZazr.zazrDescribe(dev.zazr.control.Option.some(1)));
    System.out.println("zazrNone=" + VavrVsZazr.zazrDescribe(dev.zazr.control.Option.none()));
  }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```java
package com.buildwithwaqar.patterns.functional.vavrcompare;

/** Side-by-side Option usage — VAVR (classic) vs Zazr (Java 25 sealed/records). */
public final class VavrVsZazr {
    private VavrVsZazr() {}

    public static String vavrDescribe(io.vavr.control.Option<Integer> opt) {
        return opt.map(n -> "got " + n).getOrElse("missing");
    }

    public static String zazrDescribe(dev.zazr.control.Option<Integer> opt) {
        return switch (opt) {
            case dev.zazr.control.Option.Some(var n) -> "got " + n;
            case dev.zazr.control.Option.None() -> "missing";
        };
    }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.functional.vavrcompare;
public final class Demo {
  public static void main(String[] a) {
    System.out.println("=== VAVR vs Zazr walkthrough ===");
    System.out.println("vavrSome=" + VavrVsZazr.vavrDescribe(io.vavr.control.Option.some(1)));
    System.out.println("vavrNone=" + VavrVsZazr.vavrDescribe(io.vavr.control.Option.none()));
    System.out.println("zazrSome=" + VavrVsZazr.zazrDescribe(dev.zazr.control.Option.some(1)));
    System.out.println("zazrNone=" + VavrVsZazr.zazrDescribe(dev.zazr.control.Option.none()));
  }
}
```

Walkthrough Output

```
=== VAVR vs Zazr walkthrough ===
vavrSome=got 1
vavrNone=missing
zazrSome=got 1
zazrNone=missing
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Zazr & VAVR.

Related Patterns

- Contrast Zazr & VAVR with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Zazr & VAVR → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.